TASK 22- GIT AND GITHUB VERSION CONTROL WITH COLLABORATIVE WORKFLOWS

Objective

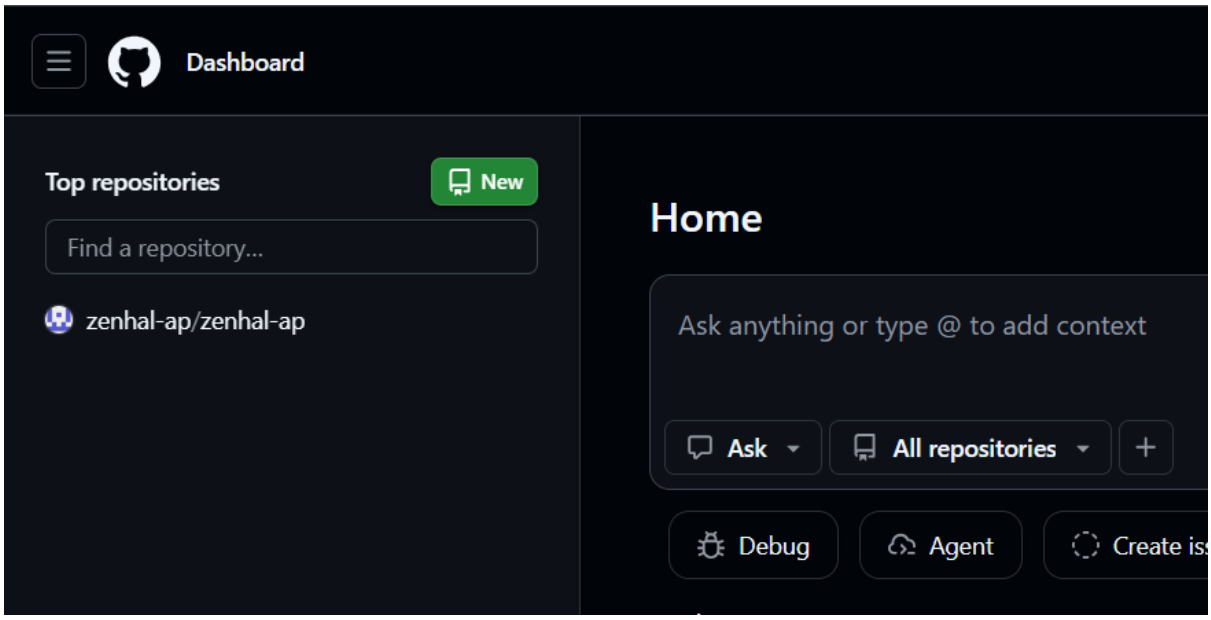
1. Sign up to GitHub and create a project repository.
2. Install and configure the Git version control system on a local machine.
3. Verify configurations by cloning the remote repository.
4. Add local files to the remote repository and vice versa (push and pull).
5. Clone a collaborator's repository and execute collaborative push and pull workflows.

1. Create a GitHub Repository and Configure Local Environment

A personal project repository was created on GitHub to host the version-controlled codebase, and Git was configured locally.

Steps:

1. Navigate to GitHub and create a new public repository named zenhal-ap.
2. Open the Git Bash terminal on the local machine.
3. Configure the global user identity and authentication credentials using Git Credential Manager (GCM) for secure browser-based token authorization.



2. Clone the Remote Repository

To verify local configurations and establish the local working copy, the repository was cloned from GitHub.

Steps:

1. Copy the repository HTTPS clone URL (https://github.com/zenhal-ap/zenhal-ap.git).
2. Run the clone command in the terminal:

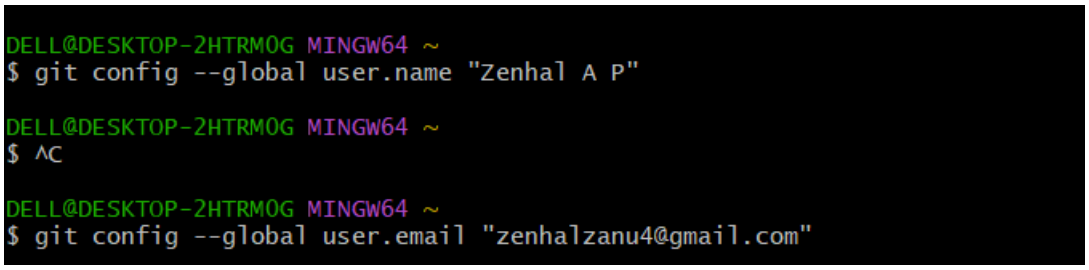
   Bash

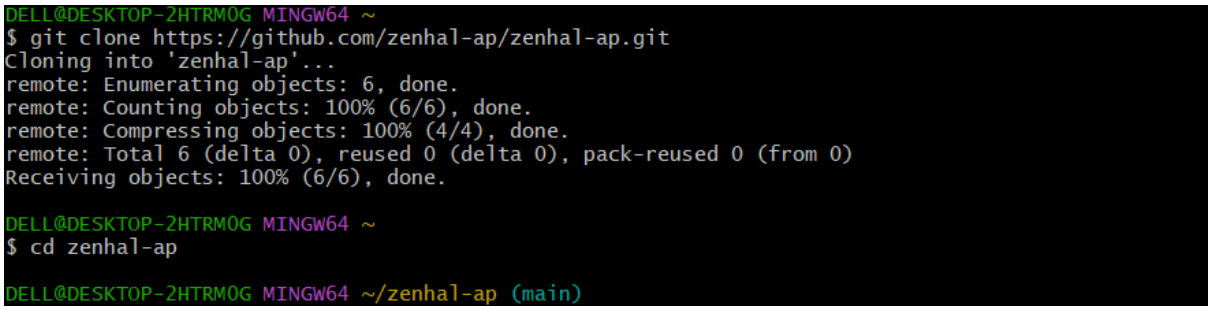
   git clone https://github.com/zenhal-ap/zenhal-ap.git

3. Navigate into the newly created repository directory:

   Bash

   cd zenhal-ap





3. Implement Local-to-Remote and Remote-to-Local Workflows (Vice Versa)

Changes were introduced locally, pushed to the remote repository, and subsequently pulled back down to verify bidirectional synchronization.

Steps for Local-to-Remote (Push):

1. Create a new local file named test.txt.
2. Stage and commit the file:

   Bash

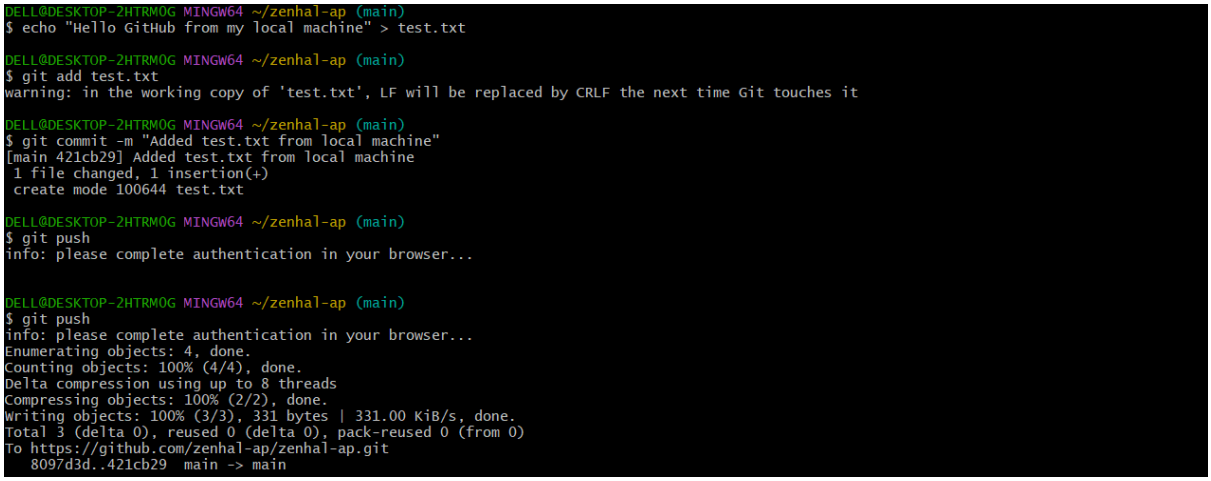
   git add test.txt
   git commit -m "Added test.txt file"

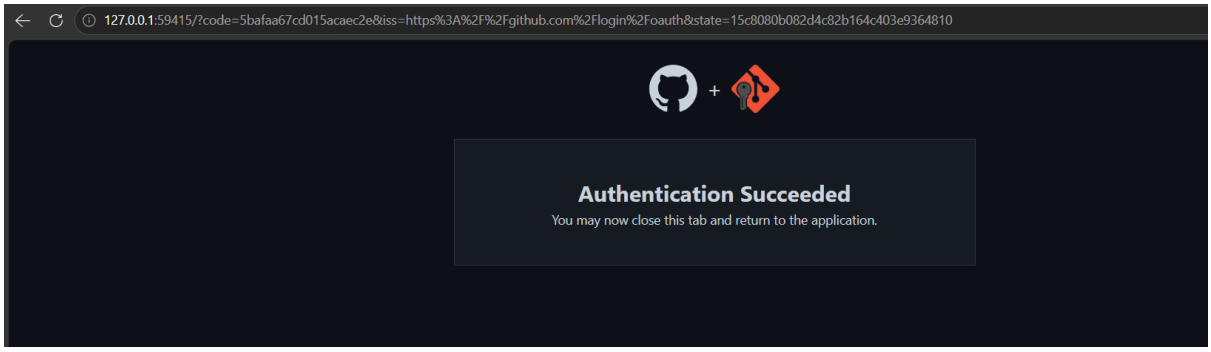
3. Push the changes to the remote main branch:

   Bash

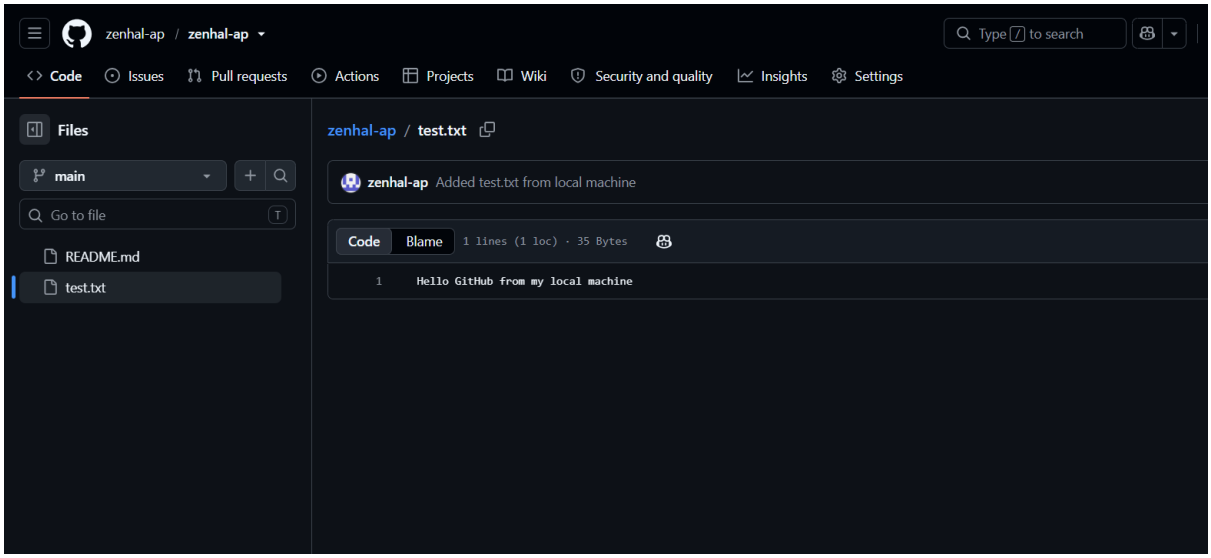
   git push

4. Authenticate via the browser prompt when requested by the Git Credential Manager. Confirm that the push completes successfully (main -> main).





In git repository:



- Steps for Remote-to-Local (Pull / Vice Versa):

1. Create a new file (remote-test.txt) directly via the GitHub web interface and commit changes to the main branch.
2. Return to the local terminal and execute the pull command:

   Bash

   git pull

3. Verify that the remote file is successfully synchronized down to the local machine via a fast-forward merge.

```
DELL@DESKTOP-2HTRM0G MINGW64 ~/zenhal-ap (main)
$ git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 980 bytes | 75.00 KiB/s, done.
From https://github.com/zenhal-ap/zenhal-ap
   421cb29..a46e2b0  main       -> origin/main
Updating 421cb29..a46e2b0
Fast-forward
 remote-test.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 remote-test.txt

DELL@DESKTOP-2HTRM0G MINGW64 ~/zenhal-ap (main)
$ ls -a
./  ../  .git/  README.md  remote-test.txt  test.txt

DELL@DESKTOP-2HTRM0G MINGW64 ~/zenhal-ap (main)
$ cat remote-test.txt
git to local machine pull
```

4. Collaborative Team Workflow

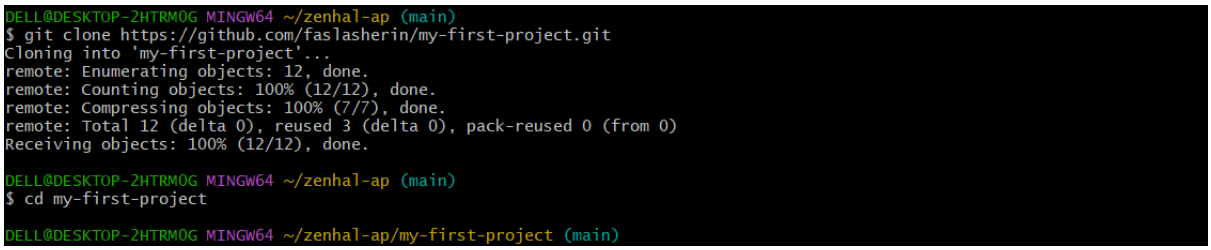
To test multi-developer team environments, participation in a shared repository was established.

Steps:

1. Accept the collaborator invitation for the shared repository (https://github.com/faslasherin/my-first-project) to gain authorized push and pull access.
2. Clone the collaborator's repository locally and navigate into its directory:

   Bash

   git clone https://github.com/faslasherin/my-first-project.git
   cd my-first-project



- Create a collaboration file locally, stage it, commit the changes, and push it up to the shared remote branch:

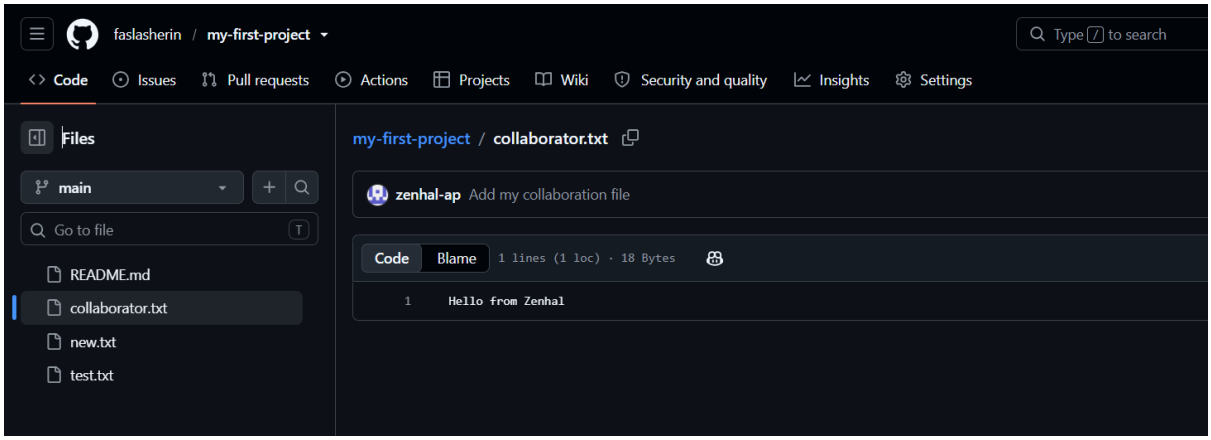
Bash
git add collaborator.txt
git commit -m "Add my collaboration file"
git push

```
DELL@DESKTOP-2HTRM0G MINGW64 ~/zenhal-ap/my-first-project (main)
$ echo "Hello from Zenhal" > collaborator.txt

DELL@DESKTOP-2HTRM0G MINGW64 ~/zenhal-ap/my-first-project (main)
$ git add collaborator.txt
warning: in the working copy of 'collaborator.txt', LF will be replaced by CRLF the next time Git touches it

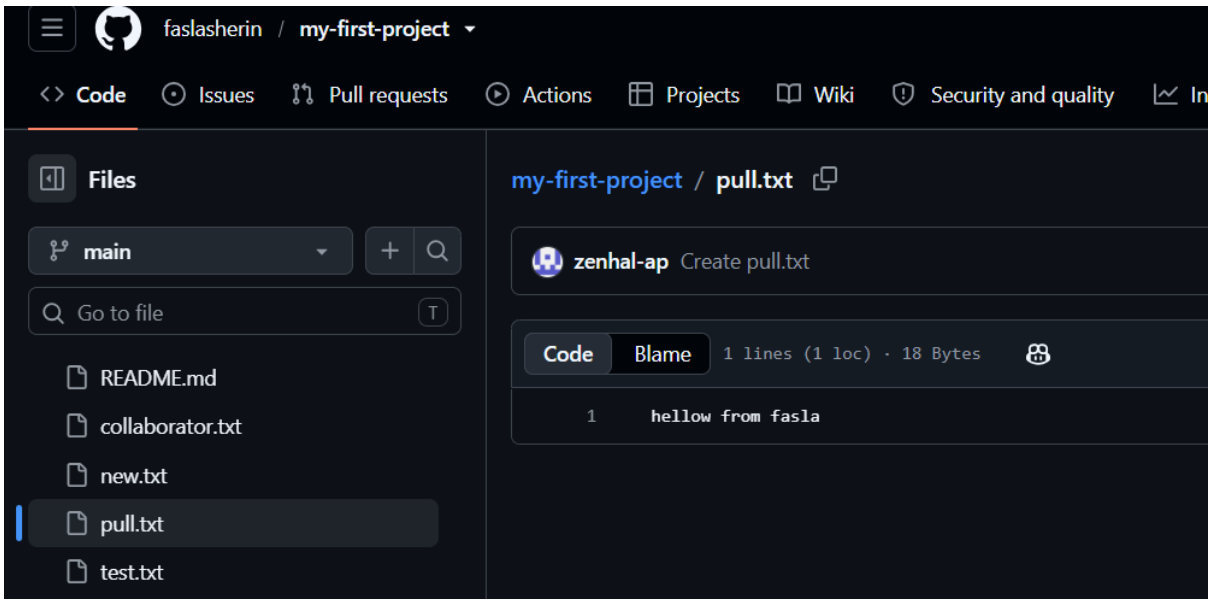
DELL@DESKTOP-2HTRM0G MINGW64 ~/zenhal-ap/my-first-project (main)
$ git commit -m "Add my collaboration file"
[main a26de52] Add my collaboration file
 1 file changed, 1 insertion(+)
 create mode 100644 collaborator.txt

DELL@DESKTOP-2HTRM0G MINGW64 ~/zenhal-ap/my-first-project (main)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 374 bytes | 374.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/faslasherin/my-first-project.git
   819e517..a26de52  main -> main
```



Test the remote-to-local synchronization loop on the shared repository by pulling down recent updates from the team environment:

Bash
git pull




```
DELL@DESKTOP-2HTRMOG MINGW64 ~/zenhal-ap/my-first-project (main)
$ git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 924 bytes | 61.00 KiB/s, done.
From https://github.com/faslasherin/my-first-project
   a26de52..c7a073b  main       -> origin/main
Updating a26de52..c7a073b
Fast-forward
 pull.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 pull.txt

DELL@DESKTOP-2HTRMOG MINGW64 ~/zenhal-ap/my-first-project (main)
$ ls
README.md  collaborator.txt  new.txt  pull.txt  test.txt

DELL@DESKTOP-2HTRMOG MINGW64 ~/zenhal-ap/my-first-project (main)
$ cat pull.txt
hellow from fasla
```

Conclusion

This task successfully demonstrated the complete lifecycle of version control management: initializing and configuring Git locally, managing personal repositories via bidirectional push and pull operations, and executing multi-user collaboration workflows on shared repositories.